

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS

CURSO: Aseguramiento de la Calidad De Software

CATEDRÁTICO: Ing. Carmelo Estuardo Mayen Monterroso

SEMESTRE: Decimo



CONTENIDO:

Guia OWASP, Investigación Planes y Casos de prueba

ALUMNO:

Carla Reneé Pérez Suriano

CARNET:

1790-21-4234

Chiquimulilla, 1 de Agosto del 2025



Guía práctica para mitigar vulnerabilidades del OWASP Top 10 – 2021



OWASP TOP 10 DE RIESGOS DE SEGURIDAD

1. Accesos de control rotos

¿Qué es?

El control de acceso constituye un mecanismo fundamental para garantizar que los usuarios operen dentro de los límites de sus permisos asignados. Este tipo de falla en el control de acceso puede permitir que usuarios no autorizados penetren sistemas, accedan a información confidencial, realicen acciones prohibidas o modifiquen datos sensibles. Las consecuencias pueden ser devastadoras, incluyendo la potencial filtración de datos confidenciales, la modificación de información crítica o la usurpación de identidad para ejecutar acciones fraudulentas suplantando la identidad de usuarios legítimos.

¿Cómo prevenirlo?

- El control de acceso debe implementar de manera impecable a nivel de dato, impidiendo que los usuarios puedan crear, leer, actualizar o eliminar cualquier dato.
- Es fundamental deshabilitar el listado de directorios del servidor web y asegurar de que los archivos de metadatos (.git) o archivos de respaldo no sean accedidos desde la raíz del sitio web.
- Los identificadores de sesiones deben ser invalidados en el servidor inmediatamente después de cerrar la sesión. Además, los tokens JWT deberían ser diseñarse con duraciones cortas para minimizar la ventana de oportunidad de posibles ataques

2. Fallas criptográficas

¿Qué es?

Las fallas criptográficas se refieren a problemas relacionados con el uso inadecuado o incorrecto de algoritmos criptográficos, la generación de claves débiles, la generación de números aleatorios que no son lo suficientemente impredecibles o la implementación incorrecta de funciones criptográficas en una aplicación.

¿Cómo prevenirlo?

- Es fundamental utilizar algoritmos de cifrado de clave pública y privada, como RSA o ECC, que son estándares de la industria altamente confiables.
- Los mecanismos de cifrado de extremo a extremo, son cruciales los datos desde el dispositivo del usuario hasta el servidor de destino, impidiendo que la comunicación puede ser interceptada.
- Los controles de acceso basados en roles son esenciales para restringir el acceso a datos confidenciales. Una regla fundamental es implementar siempre accesos y roles bajo la necesidad de saber, y con el mínimo privilegio posible.
- Realizar Pentesting periódicos para identificar y corregir fallas criptográficas, esta práctica debe incorporarse en los procesos de desarrollo, operación y mantención. No cabe duda de que invertir en un programa de ciberseguridad es más económico que hacerte cargo de las consecuencias de un ataque.

3. Ataques de inyección

¿Qué es?

son aquellos en los que se realiza la inserción de código malicioso en una aplicación a través de datos no confiables proporcionados por el usuario. Estos ataques son comunes en aplicaciones web y pueden surgir cuando la entrada del usuario no se filtra adecuadamente antes de ser utilizada por la aplicación. Las

consecuencias de inyección pueden ser devastadoras, ya que permiten a los atacantes ejecutar comandos arbitrarios en el contexto de la aplicación.

¿Cómo prevenirlo?

- **Separar los comandos de los datos:** esta técnica puede ayudar a prevenir ataques que reemplazan datos con ejecución de comandos no autorizados.
- **Consultas SQL de código con parámetros:** en lugar de estructurar un comando solo a partir del contenido ingresado por el usuario, se recomienda utilizar consultas parametrizadas (también llamadas declaraciones preparadas).
- **Eliminación del intérprete:** se puede utilizar una API segura para eliminar el intérprete.
- **Implementación de validación positiva del lado del servidor:** se sugiere emplear esta técnica con un sistema de detección de intrusiones para identificar comportamientos sospechosos del lado del cliente.

4. Diseño inseguro

¿Qué es?

Un diseño inseguro puede resultar en vulnerabilidades críticas que persisten durante toda la vida útil de la aplicación. Esto puede incluir la falta de autenticación adecuada, autorización débil, almacenamiento inseguro de datos, entre otros. Un diseño eficiente puede permitir a los atacantes explotar fácilmente las debilidades del sistema.

¿Cómo prevenirlo?

- Se debe establecer un ciclo de desarrollo seguro, respaldado por profesionales especializados en seguridad de aplicaciones, con el fin de evaluar, diseñar e implementar estrategias de protección y controles orientados a la privacidad de los datos.

- Es fundamental crear y utilizar un catálogo de patrones de diseño seguros y componentes de camino pavimentado, previamente preparados para las implementaciones eficientes.
- El modelado de amenazas debe aplicarse sistemáticamente a flujos críticos, incluyendo autenticación, control de acceso, lógica de negocio y procesos clave.
- Los equipos de desarrollo deben escribir pruebas unitarias y de integración para validar la resistencia de los flujos críticos frente al modelo de amenazas, recopilando cuidadosamente casos de uso y casos de mal uso para cada capa de la aplicación.

5. Configuración incorrecta de seguridad

¿Qué es?

La falta de configuración adecuada de los sistemas y aplicaciones para mitigar las vulnerabilidades conocidas se denomina como configuración incorrecta de seguridad. Este tipo de falla puede permitir a los atacantes acceder a información confidencial, comprometan la integridad de los datos o lleven a cabo ataques más complejos.

¿Cómo prevenirlo?

- **Endurecimiento repetible:** constituye un procedimiento que pueda ser replicado de forma rápida y eficiente de nuevos entornos adecuadamente protegidos. Los entornos de desarrollo, pruebas y producción deben configurarse de forma idéntica, utilizando credenciales distintas para cada uno.
- **Plataforma mínima:** el uso de una plataforma debe de operar sin funciones, componentes, documentación o ejemplos innecesarios. Los profesionales

deben eliminar o no instalar funciones y marcos que no se utilicen, manteniendo únicamente lo esencial para el funcionamiento del sistema.

- **Revisión y actualización de configuraciones:** Como parte del proceso de gestión de parches, es crucial realizar una tarea periódica para revisar y actualizar las configuraciones conforme a todas las notas de seguridad, actualizaciones y parches disponibles.
- **Arquitectura de aplicación segmentada:** garantiza una separación efectiva y segura entre componentes o usuarios, utilizando técnicas como segmentación, contenedores o grupos de seguridad en la nube (ACL).
- **Directivas de seguridad:** es esencial enviar directivas de seguridad a los clientes, incluyendo encabezados de seguridad que protejan la integridad del sistema.
- **Verificación automatizada:** Los equipos técnicos deben Implementar un proceso automatizado para verificar la efectividad de las configuraciones y ajustes en todos los entornos.

6. Componentes vulnerables y obsoletos

¿Qué es?

Los componentes vulnerables y obsoletos son aquellos componentes de software en una aplicación que presentan vulnerabilidades conocidas o se encuentran desactualizados. Estos componentes pueden incluir bibliotecas de terceros, módulos, o incluso sistemas operativos subyacentes.

Las vulnerabilidades asociadas con estos componentes pueden ser aprovechadas por atacantes para comprometer la seguridad de la aplicación y, potencialmente de todo el sistema.

¿Cómo prevenirlo?

- Eliminar las dependencias no son utilizadas, funcionalidades, componentes, archivos y documentación innecesarios.

- Al obtener componentes, es fundamental recurrir únicamente a fuentes oficiales a través de enlaces seguros. Se recomienda priorizar paquetes firmados, ya que hay una posibilidad de incorporar componentes maliciosos que hayan sido modificados
- La supervisión constante de bibliotecas y componentes es esencial, especialmente aquellos que carecen de mantenimiento activo o no generan parches de seguridad para versiones anteriores.

7. Fallas de identificación y Autenticación

¿Qué es?

La incapacidad de un sistema para verificar correctamente la identidad de un usuario y garantizar que solo los usuarios autorizados puedan acceder a recursos protegidos se conoce como fallas de identificación y autenticación.

¿Cómo prevenirlo?

- Siempre que sea posible, se debe implementar autenticación multifactor con el objetivo prevenir ataques automatizados de relleno de credenciales, fuerza bruta y reutilización de credenciales robadas.
- Los procesos de registro, recuperación de credenciales y las rutas API deben estar fortalecidos contra ataques de enumeración de cuentas, utilizando mensajes para todos los resultados.
- Se debe emplear un administrador de sesiones seguro y basado en el servidor, que genere un nuevo ID de sesión aleatorio con alta entropía tras el inicio de sesión. Dicho identificador no debe incluirse en la URL, debe almacenarse de forma segura y ser invalidado al cerrar sesión.

8. Fallas de integridad de datos y software

¿Qué es?

Las fallas de integridad de datos y software ocurren cuando un sistema no logra proteger adecuadamente la integridad de los datos almacenados o del propio software. Estos fallos pueden incluir la falta de controles de integridad en los datos, vulnerabilidades que facilitan la manipulación de datos o permiten la ejecución de código malicioso dentro de una aplicación.

¿Cómo prevenirlo?

- **Verificación de Firmas Digitales:** Es fundamental que las organizaciones garanticen que todo el software y las actualizaciones provengan de fuentes confiables y estén firmados digitalmente para verificar su autenticidad.
- **Implementación de Controles de Integridad:** Los equipos de seguridad deben utilizar hashes o firmas digitales para verificar la integridad de los archivos críticos antes de implementarlos en entornos de producción, minimizando así potenciales riesgos de manipulación.
- **Auditorías Regulares:** Es crucial realizar auditorías periódicas del software y los datos para detectar cualquier alteración no autorizada, estableciendo un protocolo sistemático de monitoreo y revisión continua.
- **Control Estricto de la Cadena de Suministro de Software:** Las organizaciones deben evaluar rigurosamente a todos los proveedores de software, asegurándose de que cumplan con las mejores prácticas de seguridad y mantengan estándares de calidad que protejan la infraestructura tecnológica.

9. Fallas de monitoreo y registro de seguridad

¿Qué es?

Las fallas de monitoreo y registro de seguridad se refieren a la falta de capacidades adecuadas para monitorear y registrar eventos de seguridad dentro de una aplicación.

La ausencia de un monitoreo efectivo puede permitir a los atacantes operar sin ser detectados, lo que conduciría a graves consecuencias para la seguridad y la integridad de la aplicación. Estas vulnerabilidades pueden retrasar la detección y respuesta a incidentes de seguridad, haciendo que los atacantes operen durante más tiempo y causen un mayor daño.

¿Cómo prevenirlo?

- Se recomienda que los equipos de seguridad implementen estrategias exhaustivas para el registro de eventos críticos. Deben asegurar que todos los errores relacionados con inicio de sesión, control de acceso y validación de entradas de datos del lado del servidor
- Es fundamental que los registros se generen en un formato que facilite su procesamiento por herramientas de gestión de registros, garantizando su utilidad y accesibilidad. Además, es crucial que los datos de los registros estén correctamente codificados para prevenir posibles inyecciones o ataques dirigidos al sistema de monitoreo.
- Para transacciones de alto valor, se debe implementar una traza de auditoría robusta con controles de integridad que impidan su modificación o eliminación

10. Falsificación de solicitudes del lado del servidor

¿Qué es?

La falsificación de solicitudes del lado del servidor surge cuando una aplicación web solicita recursos remotos sin validar las URLs proporcionadas por el usuario. Un atacante puede inducir a los servidores web a realizar solicitudes a otros

recursos en su nombre, permitiéndole acceder a recursos internos o externos que normalmente estarían fuera de su alcance.

La gravedad de estas vulnerabilidades también se ha intensificado debido a la adopción de servicios en la nube y la complejidad de las arquitecturas actuales.

¿Cómo prevenirlo?

- El desarrollador debe sanitizar y validar meticulosamente todos los datos de entrada proporcionados por el cliente, garantizando su integridad y seguridad antes de procesarlos.
- Es fundamental implementar un esquema de control estricto para URL, puertos y destinos, utilizando una lista positiva de elementos permitidos que restrinja el acceso únicamente a recursos previamente autorizados.
- Es crucial evitar enviar respuestas en formato "crudo" a los clientes, priorizando la presentación de información procesada y estructurada que facilite su comprensión.

GLOSARIO

1. **ACL (Access Control List):** Lista que define qué usuarios o sistemas tienen permiso para acceder a determinados recursos.
2. **Autenticación multifactor (MFA):** Metodo de autenticación que requiere más de un factor para verificar la identidad del usuario
3. **Criptografía:** Técnica que permite proteger la información mediante el uso de algoritmos matemáticos para cifrar y descifrar datos.
4. **Pentesting:** Simulación controlada de un ataque para identificar vulnerabilidades en sistemas y aplicaciones.

5. **Hash:** Función criptográfica que convierte datos en una cadena alfanumérica única, usada para verificar integridad.
- .
6. **Auditoria:** Revisión sistema de sistemas, procesos y datos para asegurar que se cumplen las políticas de seguridad

SEGUNDA PARTE:

Realizar una investigación sobre los siguientes temas.

- **¿Qué son los Planes de Prueba?**

El Plan de pruebas describe el alcance, el enfoque, los recursos y el cronograma de todas las actividades de prueba. Identifica los elementos y características que se van a probar, es decir, los tipos de pruebas. Contiene una estrategia detallada y ejecutable para la realización. Define el objetivo detallado de la prueba, específico de un sistema en particular, el enfoque de prueba, el entorno de prueba, las condiciones de prueba y el plan de prueba.

- **¿Qué son los Casos de Prueba?**

Es un conjunto de condiciones o variables bajo las cuales se determinará si una aplicación, un sistema de software o una característica o comportamiento de estos resulta o no aceptable. Se pueden realizar muchos casos de prueba para determinar que un requisito es completamente satisfactorio. Con el propósito de comprobar que todos los requisitos de una aplicación son revisados, debe haber al menos un caso de prueba para cada requisito a menos que un requisito tenga requisitos secundarios.

- Ejemplos:

Plan de Pruebas

Plan de pruebas de calidad en la gestión de proyectos

This slide covers the project quality testing table for documenting any quality compromising issues. It also includes testing information such as expected results, actual results, tester, etc.

Date	Test Description	Expected Results	Actual Results	Pass/Fail	Tested by	Tester Comments
10/01/2022	Website login	Should land to home screen	User directed to other page	Fail	Henry	Contact web development team for changes
13/01/2022	Click magnifying glass	Whole page gets zoomed in	Text size increases	Fail	George	Contact web development team for changes
14/01/2022	Signing up for website	Get confirmation email	Confirmation email received	Pass	Emma	No issues
Add test here	Add test here	Add test here	Add test here	Add test here	Add test here	Add test here
Add test here	Add test here	Add test here	Add test here	Add test here	Add test here	Add test here
Add test here	Add test here	Add test here	Add test here	Add test here	Add test here	Add test here
Add test here	Add test here	Add test here	Add test here	Add test here	Add test here	Add test here

This slide is 100% editable. Adapt it to your need and capture your audience's attention.

Casos de pruebas

Informe de errores de casos de prueba del proyecto

Name: ABC Systems

Project ID: W 26938

Application ID/Name: ABC CRM Software

From Report Date: 17 June 2020

Report Date: 19 June 2020

Complete By(Milestone): 21 July 2020

Manager: Adam James

QA Manager: William Nichols

Summary		
Total Test Cases		3
Executed		2
Pass		1
Fail		1
Not Executed		1

Functional Testing

Total Case ID	Description	Pass/Fail/Not Executed	Test Date	Responsible Developer	Responsible Tester	Comment	Additional Comment (Other than QA Team)
01	Valid Login	Pass	18-june-2020	Developer 1	Tester 1	Login Successful	Add Text Here
02	Login Error on Invalid Password	Fail	18-june-2020	Developer 1	Tester 2	Incorrect Error Message on Failure	Add Text Here
03	Add Text Here	Not Executed	18-june-2020	Developer 3	Tester 2	NA	Add Text Here

This slide is 100% editable. Adapt it to your needs and capture your audience's attention.

Plan de Pruebas para un Catálogo

Objetivo: Este plan de pruebas tiene como objetivo, validar de manera integral el correcto funcionamiento de create, Read, Update y Delete dentro de un catálogo de productos. Esta validación permitirá asegurar que los usuarios puedan gestionar los productos de forma confiable y segura, sin errores visibles ni fallas en la lógica del sistema o en la persistencia de los datos

Descripción del Módulo

El módulo a evaluar forma parte de una aplicación web orientada a la gestión de un catálogo de productos deportivos. Este catálogo permite a los usuarios realizar operaciones básicas sobre los productos, tales como:

- **Crear** nuevos registros con información como nombre del producto, categoría, precio y cantidad.
- **Editar** productos existentes para modificar cualquiera de sus atributos.
- **Eliminar** productos de forma permanente del sistema.

Este tipo de funcionalidades es esencial en aplicaciones de tipo administrativo o de inventario, ya que constituyen el núcleo de la gestión de datos.

Alcance del Plan de Pruebas

Incluye:

- Validación de funcionalidades del CRUD.
- Comprobación del comportamiento de la interfaz en cada operación.
- Evaluación del manejo de errores y validaciones del lado del cliente y del servidor.

Excluye:

- Pruebas de rendimiento o carga.
- Pruebas de seguridad como inyecciones o accesos no autorizados.
- Pruebas de integración con otros módulos o servicios externos

Recursos y Herramientas

Para ejecutar el plan de pruebas, se requiere el siguiente conjunto de herramientas y entornos:

- **Entorno de desarrollo o pruebas:** PostgreSQL
- **Navegador web:** Google Chrome,
- **Herramienta para pruebas manuales de API:** Postman
- **Editor de código:** Visual Studio Code.
- **Base de datos con datos de prueba.**